# MC458 - Projeto e Análise de Algoritmos I

Prof. Dr. Hilder Vitor Lima Pereira

Instituto de computação

UNICAMP

07 de abril de 2026

1 Heap Sort
- Motivação

2 Cotas inferiores
- Cota inferior para ordenação
- Cota inferior para busca em vetor ordenado

3 Perguntas ou comentários?

# Heap Sort

# Heap Sort:

## Motivação

# Selection sort

**SelectionSort**$(v, n)$

**Input:** $v \in \mathbb{N}^n$

**for** $1 \leq i < n$ **do**

  $i_{max} \leftarrow \texttt{FindMaximum}(v, 0, n - i)$

  $v[i_{max}] \leftrightarrow v[n - i]$

Gostaríamos de executar $\texttt{FindMaximum}(v, 0, n - i)$ em $o(n)$.

# Max-heaps

## Definição

Um heap $v[1 \ldots n]$ é chamado de max-heap se cada nó for uma folha ou tiver valor maior que seus filhos, i.e., para $1 \leq i \leq n/2$,

- $v[i] \geq v[2i]$ e $v[i] \geq v[2i+1]$

# Consertando um max-heap

## Problema

Temos um vetor $v$ com $n$ nós e um nó $k$ que não respeita a propriedade de max-heap, ou seja, é menor do que algum dos filhos, mas cada um dos filhos de $k$ é um max-heap.
Como consertar $v$ para que $k$ seja um max-heap?

# MaxHeapify

Ideia: consertar a raiz, possivelmente introduzindo um problema a esquerda ou a direita, então consertar recursivamente o que ficou problemático.

# MaxHeapify

Ideia: consertar a raiz, possivelmente introduzindo um problema a esquerda ou a direita, então consertar recursivamente o que ficou problemático.

---

`MaxHeapify`$(v, n, k)$

---

**Input:** $v \in \mathbb{N}^n, k \in \mathbb{N} : 1 \leq k \leq n$

$e \leftarrow 2k$
$d \leftarrow 2k + 1$
$i_{max} \leftarrow k$
**if** $e \leq n$ *and* $v[e] > v[i_{max}]$ **then**
  $i_{max} \leftarrow e$
**if** $d \leq n$ *and* $v[d] > v[i_{max}]$ **then**
  $i_{max} \leftarrow d$
**if** $i_{max} \neq k$ **then**
  $v[k] \leftrightarrow v[i_{max}]$
  `MaxHeapify`$(v, n, i_{max})$

---

# Construindo um max-heap

- cada folha já é um max-heap (com um único nó)
- consertamos o penúltimo nível
- depois o antepenúltimo
- e assim por diante

---

`BuildMaxHeap`$(v, n)$

---

**Input:** $v \in \mathbb{N}^n$

1 $i \leftarrow \lfloor n/2 \rfloor$

2 **while** $i \geq 1$ **do**

3 $\quad$ `MaxHeapify`$(v, n, i)$

4 $\quad$ $i \leftarrow i - 1$

---

# Análise de BuildMaxHeap

```
BuildMaxHeap(v, n)
Input: v ∈ ℕ^n
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

## Terminação

$\forall n \in \mathbb{N}^*$ `BuildMaxHeap`$(v, n)$ termina

# Análise de BuildMaxHeap

```
BuildMaxHeap(v, n)
Input: v ∈ ℕ^n
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

## Terminação

$\forall n \in \mathbb{N}^*$ `BuildMaxHeap`$(v, n)$ termina

## Invariante

No início da $i$-ésima iteração, $i+1, \ldots, n$ são raízes de max-heaps.

# Análise de BuildMaxHeap

**BuildMaxHeap**$(v, n)$

**Input:** $v \in \mathbb{N}^n$

```
i ← ⌊n/2⌋
while i ≥ 1 do
    MaxHeapify(v, n, i)
    i ← i − 1
```

## Terminação

$\forall n \in \mathbb{N}^*$ BuildMaxHeap$(v, n)$ termina

## Invariante

No início da $i$-ésima iteração, $i + 1, \ldots, n$ são raízes de max-heaps.

## Teorema

$\forall n \in \mathbb{N}^*$ BuildMaxHeap$(v, n)$ está correto

# Complexidade de `BuildMaxHeap`

Quantas operações elementares o algoritmo `BuildMaxHeap` faz?

- Temos $O(n)$ iterações

# Complexidade de `BuildMaxHeap`

Quantas operações elementares o algoritmo `BuildMaxHeap` faz?

- Temos $O(n)$ iterações
- Cada iteração chama `MaxHeapify`$(v, n, i)$

# Complexidade de BuildMaxHeap

Quantas operações elementares o algoritmo BuildMaxHeap faz?

- Temos $O(n)$ iterações
- Cada iteração chama MaxHeapify$(v, n, i)$
- MaxHeapify$(v, n, i)$ roda em $O(h)$, onde $h$ é a altura do heap

# Complexidade de `BuildMaxHeap`

Quantas operações elementares o algoritmo `BuildMaxHeap` faz?

- Temos $O(n)$ iterações
- Cada iteração chama `MaxHeapify`$(v, n, i)$
- `MaxHeapify`$(v, n, i)$ roda em $O(h)$, onde $h$ é a altura do heap
- Então, no total, temos $O(n \cdot h) = O(n \log n)$

# Complexidade de `BuildMaxHeap`

Análise mais cuidadosa:

- seja $h$ a altura da árvore inteira
  - temos 1 nó de altura $h$
  - temos 2 nós de altura $h - 1$
  - temos 4 nós de altura $h - 2$
  - e assim por diante
- uma chamada de `MaxHeapify` leva tempo $O(h)$
- vamos somar os tempos de todas as chamadas

# Complexidade de `BuildMaxHeap` (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

# Complexidade de `BuildMaxHeap` (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

# Complexidade de `BuildMaxHeap` (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

# Complexidade de `BuildMaxHeap` (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

# Complexidade de `BuildMaxHeap` (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\sum_{i=1}^{h} \frac{i}{2^i} =$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}\sum_{i=1}^{h} \frac{i}{2^i} = & \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \\ & + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\end{aligned}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \\ + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \\ + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\end{aligned}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = & \left.\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} \quad 1 - \frac{1}{2^h} < 1 \\
& + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \\
& + \frac{1}{2^3} + \cdots + \frac{1}{2^h}
\end{aligned}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} & \Bigg\} \quad 1 - \frac{1}{2^h} < 1 \\
+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} & \Bigg\} \quad \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
+ \frac{1}{2^3} + \cdots + \frac{1}{2^h} &
\end{aligned}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} \quad & 1 - \frac{1}{2^h} < 1 \\
+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} \quad & \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
+ \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} \quad & \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Bigg\} \quad & 1 - \frac{1}{2^h} < 1 \\
+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Bigg\} \quad & \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
+ \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Bigg\} \quad & \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

Ou seja,

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} &\Big\} \quad 1 - \frac{1}{2^h} < 1 \\
+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} &\Big\} \quad \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
+ \frac{1}{2^3} + \cdots + \frac{1}{2^h} &\Big\} \quad \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

Ou seja,

$$T(n) = c2^h \sum_{i=0}^{h-1} \frac{1}{2^i}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = & \left.\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & 1 - \frac{1}{2^h} < 1 \\
& \left.+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
& \left.+ \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

Ou seja,

$$T(n) = c2^h \sum_{i=0}^{h-1} \frac{1}{2^i} \leq c2^h \cdot \sum_{i=0}^{\infty} \frac{1}{2^i}$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = & \left.\frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & 1 - \frac{1}{2^h} < 1 \\
& \left.+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
& \left.+ \frac{1}{2^3} + \cdots + \frac{1}{2^h}\right\} & \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

Ou seja,

$$T(n) = c2^h \sum_{i=0}^{h-1} \frac{1}{2^i} \leq c2^h \cdot \sum_{i=0}^{\infty} \frac{1}{2^i} = c2^h \cdot 2$$

# Complexidade de BuildMaxHeap (cont)

Seja $h = \lfloor \log_2 n \rfloor$ a altura da árvore inteira, então

$$T(n) = \sum_{i=1}^{h} 2^{h-i} \cdot c \cdot i = c \cdot 2^h \sum_{i=1}^{h} \frac{i}{2^i}$$

Note que

$$\begin{aligned}
\sum_{i=1}^{h} \frac{i}{2^i} = \frac{1}{2} + \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} & \quad 1 - \frac{1}{2^h} < 1 \\
+ \frac{1}{2^2} + \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} & \quad \frac{1}{2} - \frac{1}{2^h} < \frac{1}{2} \\
+ \frac{1}{2^3} + \cdots + \frac{1}{2^h} \Big\} & \quad \frac{1}{4} - \frac{1}{2^h} < \frac{1}{4}
\end{aligned}$$

Ou seja,

$$T(n) = c2^h \sum_{i=0}^{h-1} \frac{1}{2^i} \leq c2^h \cdot \sum_{i=0}^{\infty} \frac{1}{2^i} = c2^h \cdot 2 = O(n)$$

# Chegando na ordenação...

Usando `BuildMaxHeap`, conseguimos transformar um vetor $v \in \mathbb{Z}^n$ desordenado em um max-heap em tempo $O(n)$
Como podemos usar um max-heap para ordernar?

# O algoritmo HeapSort

```
HeapSort(v, n)
Input: v ∈ ℕ^n
BuildMaxHeap(v, n)
i ← n
while i ≥ 2 do
    ▷ Coloca máximo na i-ésima posição
    v[1] ↔ v[i]
    ▷ Arruma raíz (nó 1) de heap v com i − 1 nós
    MaxHeapify(v, i − 1, 1)
    i ← i − 1
```

**Invariante**

No início de cada iteração vale

1. $v[1 \ldots i]$ é um max-heap
2. $v[i+1 \ldots n]$ é vazio ou crescente
3. $i = n$ ou $v[1 \ldots i] \leq v[i+1]$

# Complexidade de HeapSort

Complexidade temporal:

- `BuildMaxHeap`: $O(n)$
- Cada chamada de `MaxHeapify`: $O(\log n)$ no pior caso
- Temos $n$ dessas chamadas: $O(n \log n)$
- Total no pior caso: $T(n) \in O(n + n \log n) = O(n \log n)$

Complexidade espacial:

# Complexidade de HeapSort

Complexidade temporal:

- `BuildMaxHeap`: $O(n)$
- Cada chamada de `MaxHeapify`: $O(\log n)$ no pior caso
- Temos $n$ dessas chamadas: $O(n \log n)$
- Total no pior caso: $T(n) \in O(n + n \log n) = O(n \log n)$

Complexidade espacial: $O(1)$

Conseguimos o mesmo tempo que o Merge Sort, mas sem usar memória auxiliar!